

Intelligent Support Ticket Classification with Retrieval-Augmented Generation (RAG)

System Analysis Document

1. Introduction

Customer support teams handle a continuous stream of tickets that must be read, categorized, and answered accurately and consistently. When this process relies entirely on manual effort, agents spend significant time searching for relevant precedents and drafting responses, which slows resolution times and increases the risk of inconsistent answers, particularly for organizations handling a high ticket volume.

This project proposes an AI-assisted support ticket handling system intended for use by support agents, not customers. Upon receiving a new ticket, the system automatically classifies it, retrieves similar historical tickets and relevant knowledge base documents using ChromaDB-based vector similarity search, and generates a suggested response through Retrieval-Augmented Generation (RAG). The support agent remains in full control of the interaction: the AI-generated response is treated as a draft that the agent reviews, edits if necessary, and approves before it is sent to the customer. This design combines the speed and consistency of NLP-based automation with the judgment and accountability of a human agent.

2. Use Case Diagram

The use case diagram identifies the actors that interact with the system and the main functions each actor can perform. It distinguishes between the day-to-day ticket-handling activities carried out by the Support Agent and the administrative and maintenance activities carried out by the Administrator, while also showing which use cases depend on the underlying classification and retrieval logic.

[Insert Use Case Diagram Here]

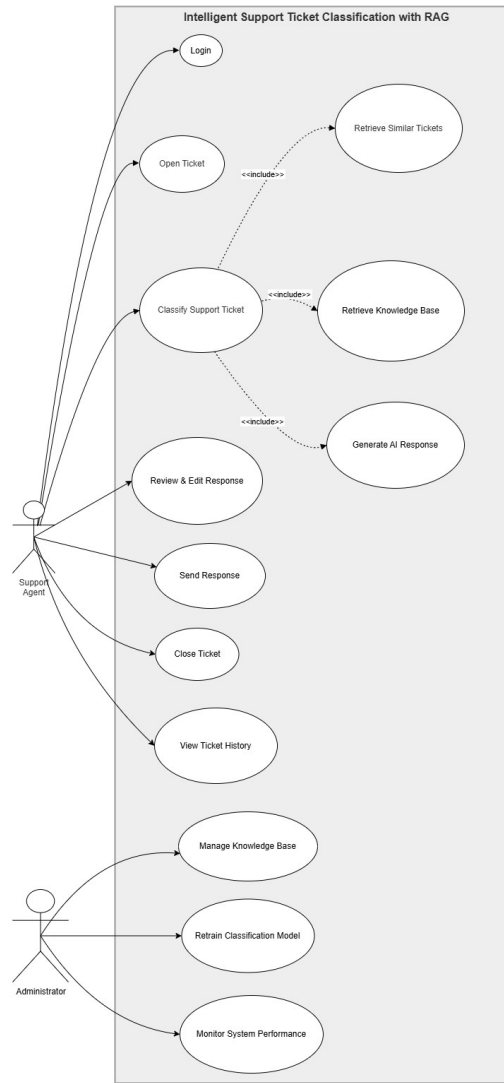


Figure 1: Use Case Diagram – Intelligent Support Ticket Classification with RAG

Two actors are defined. The Support Agent logs into the system, opens tickets, and classifies a support ticket; classifying a ticket includes retrieving similar tickets, retrieving knowledge base content, and generating an AI response, since these steps are necessary sub-processes of classification and cannot occur independently. The agent then reviews and edits the suggested response, sends the final response, closes the ticket, and may view ticket history. The Administrator is responsible for system upkeep: managing the knowledge base, retraining the classification model, and monitoring overall system performance. Separating these responsibilities keeps the operational workflow (agent) distinct from the maintenance workflow (administrator).

3. Sequence Diagram

The sequence diagram describes the runtime interaction that occurs when a Support Agent requests AI assistance for an open ticket. It shows the order in which the User Interface (UI) coordinates calls to the Classification Model, the Embedding Model, the Vector Database (ChromaDB), the Knowledge Base, and the RAG Generator to produce a suggested response.

[Insert Sequence Diagram Here]

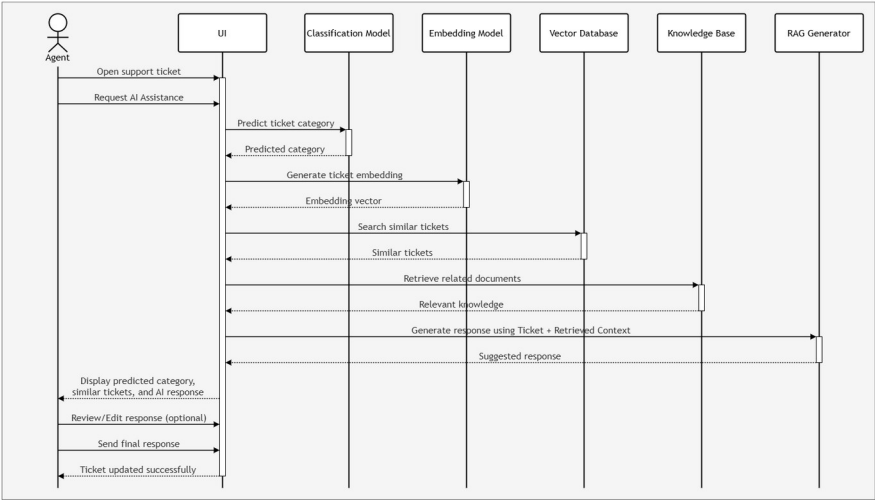


Figure 2: Sequence Diagram – AI-Assisted Ticket Response Flow

After the agent opens a ticket and requests AI assistance, the UI first sends the ticket text to the Classification Model, which returns a predicted category. The UI then forwards the ticket text to the Embedding Model, which converts it into a numerical vector representation. This embedding is passed to the Vector Database, which searches the ChromaDB collection for similar historical tickets and returns them to the UI. In parallel, the UI queries the Knowledge Base to retrieve documents relevant to the ticket's content. Finally, the UI sends the original ticket together with the retrieved tickets and knowledge base context to the RAG Generator, which produces a suggested response. The UI displays the predicted category, similar tickets, and AI-generated response to the agent, who may edit the text before sending the final response, at which point the ticket is updated.

4. System Architecture

The system architecture organizes the solution into distinct functional layers, from the agent-facing front end through the NLP and retrieval components to the final output stage. This layered structure separates concerns clearly and allows each component to be developed, tested, and scaled independently.

[Insert System Architecture Diagram Here]

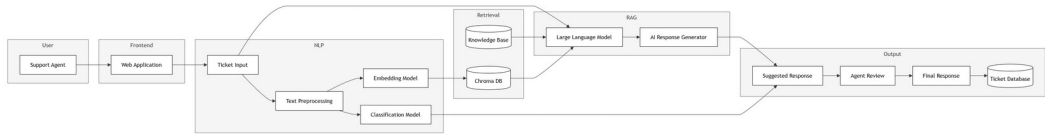


Figure 3: System Architecture Diagram

The Support Agent interacts with the system through a Web Application (Frontend layer), which submits ticket text as input to the NLP layer. Within this layer, Text Preprocessing cleans and normalizes the ticket text before it is passed to two parallel components: the Classification Model, which predicts the ticket category, and the Embedding Model, which converts the ticket into a vector representation for semantic search. The embedding is sent to the Retrieval layer, where the ChromaDB vector store returns similar past tickets and the Knowledge

Base supplies relevant support documentation. Both retrieval outputs feed into the RAG layer, where a Large Language Model combines the ticket, the classification, and the retrieved context to produce an AI Response Generator output. This suggested response, along with the predicted category, is passed to the Output layer, where the agent performs Agent Review before the Final Response is stored in the Ticket Database. Data therefore flows in a single coherent pipeline: from raw ticket text, through classification and retrieval, into generation, and finally into a human-approved reply.

5. Conclusion

The proposed architecture improves support ticket handling by combining NLP-based classification with ChromaDB-driven retrieval and RAG-based response generation. Automatic classification reduces manual triage effort, retrieval of similar tickets and knowledge base content grounds the generated response in relevant, verified information, and the RAG generator produces a coherent draft reply in place of a blank page. Crucially, the Support Agent retains full responsibility for the final response through a mandatory review and edit step, ensuring that automation accelerates the workflow without removing human accountability from customer communication. Overall, the system demonstrates how modern NLP and generative AI techniques can be applied responsibly to a real-world support operations problem.